

EXPERIMENT 4 — MISSIONARIES AND CANNIBALS

AIM

To write a Python program to solve the **Missionaries and Cannibals problem using Breadth-First Search (BFS)**.

ALGORITHM

1. Start with **3 Missionaries, 3 Cannibals, and the boat on the left bank.**
2. Represent the state as **(Missionaries, Cannibals, Boat)**.
3. Generate possible boat movements.
4. Check whether each new state is safe.
5. Avoid already visited states.
6. Store valid states in a queue.
7. Continue until all missionaries and cannibals reach the right bank.
8. Display the solution path.
9. Stop.

PYTHON PROGRAM

```
from collections import deque

start = (3, 3, 0)
goal = (0, 0, 1)

def valid(m, c):
    return (0 <= m <= 3 and 0 <= c <= 3 and
            (m == 0 or m >= c) and
            (3-m == 0 or 3-m >= 3-c))

queue = deque([(start, [])])
visited = {start}

while queue:
    state, path = queue.popleft()
    m, c, boat = state

    if state == goal:
        print("Solution:")
        for s in path + [state]:
            print(s)
        break

    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

    for dm, dc in moves:
        if boat == 0:
            nm = m - dm
            nc = c - dc
            nb = 1
        else:
            nm = m + dm
            nc = c + dc
```

```

        nb = 0

    new_state = (nm, nc, nb)

    if valid(nm, nc) and new_state not in visited:
        visited.add(new_state)
        queue.append(
            (new_state, path + [state])
        )

```

OUTPUT:

```

=====
Solution:
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)
.

```

RESULT:

Thus, the **Missionaries and Cannibals** problem was successfully solved using **Breadth-First Search** in **Python**